Course Code	Course Title	L	T	P	C
MACSE501	**Data Structures and Algorithm Analysis**	**3**	**0**	**2**	**4**
Pre-requisite	Nil	**Syllabus Version**			
		1			

Course Objectives

1. To familiarize the concepts of data structures and provide a mathematical framework for the design and analysis of algorithms.
2. To provide a deeper insight into linear and non-linear data structures.
3. To disseminate knowledge on how to develop algorithms using various design strategies to solve real-world problems.

Course Outcomes

1. Develop the ability to analyse the efficiency of algorithms using asymptotic notations and implement linear data structures.
2. Apply tree and graph-based data structures to design and implement efficient traversal, search, and pathfinding algorithms.
3. Demonstrate the ability to design and implement efficient solutions to complex problems using divide-and-conquer and greedy strategies.
4. Formulate optimal and exhaustive search-based solutions for computationally intensive problems using dynamic programming, backtracking, and branch-and-bound technique.
5. Critically analyse computational complexity, differentiate between P, NP, and NP-complete problems, and apply approximation algorithms to develop near-optimal solutions for intractable problems.

Module:1	**Complexity Analysis and Elementary Data Structures**	**8 hours**

Overview and importance of algorithms and data structures- Algorithm specification, Asymptotic Notation - The Big-O, Omega and Theta notation, Recursion, Performance analysis, Array, Stack, Queue, Linked-list and its types, various representations, operations and applications of Linear Data Structures.

Module:2	**Non Linear Data Structures**	**10 hours**

Trees: Binary trees, Properties of binary trees, Binary Search Tree, AVL, Red-Black Trees, B- Trees, Segment Trees - Heaps: Binary heaps, Min-Max heap – Graphs: Representation, Breadth first Search, Depth First Search, Shortest path algorithm –Bellman Ford algorithm - Network flow algorithm: Ford-Fulkerson and Push-relabel algorithm.

Module:3	**Divide and Conquer and Greedy Techniques**	**8 hours**

Divide and Conquer: Merge Sort, Quick Sort, Karatsuba's fast multiplication method, Strassen's Matrix Multiplication - Greedy Strategy: Job Sequencing Problem with Deadlines, Minimum Spanning Tree: Prims algorithm, Kruskal's algorithm, Huffman coding.		
Module:4	**Dynamic Programming and Backtracking Branch and Bound Techniques**	**9 hours**
Dynamic programming: Matrix Chain Multiplication, Longest Common Subsequence - Backtracking: N-Queens problem, Subset Sum problem, Graph coloring problem Branch and Bound: Subset sum, A-Star, LIFO-BB and FIFO BB methods.		
Module:5	**Complexity Classes and Approximation Algorithms**	**8 hours**
Class P - Class NP - Reducibility and NP-completeness - Circuit Satisfiability problem- 3CNF, Independent Set, Clique, Approximation Algorithms: Vertex Cover, Set Cover and Travelling salesman.		
Module:6	**Contemporary Issues**	**2 hours**
	Total Lecture Hours:	**45 hours**

Text Book(s)

Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. , "**Introduction to Algorithms** ", MIT Press, **2022**
Allen, Weiss Mark. , "**Data structures and algorithm analysis in C++**", Pearson Education India, **2007**

Reference Books

Skiena, Steven S. , "**The Algorithm Design Manual (Texts in Computer Science)**", Springer, 3rd Edition, **2020**
Jon Kleinberg and EvaTardos, "**Algorithm Design**", Pearson Education, 1st Edition, **2014**
Brass, Peter., "**Advanced data structures**", Cambridge University Press, **2008**

Indicative Experiments

1. Linear data structures: stack, queue, Linked lists In the situation where there are multiple users or a network computer system, you probably share a printer with other users. When you request to print a file, your request is added to the print queue. When your request reaches the front of the print queue, your file is printed. This ensures that only one person at a time has access to the printer and that this access is given on a first-come, first served basis. Design and implement the algorithm for this scenario.	4 hours

2. Non-linear data structures: Trees & Graphs You are making an iPod playlist to hear the songs. Assuming that shuffle functions are not applicable, choose an appropriate data structure that will add and delete songs onto you're your iPod in such a way that the recently inserted song will always be the first song currently on the iPod.	4 hours
3. Graph algorithms: BFS, DFS, network flow You are developing a diagnostic tool for a large, complex communication network (e.g., an internet service provider's infrastructure, or a peer-to-peer file-sharing network). The network shown below is an undirected, unweighted graph, where devices are nodes (routers, servers, end-points) which are connected directly by edges. Nodes: A, B, C, D, E, F, G, H, I, J Edges: A-B, A-C B-D, B-E C-E, C-F, D-G, E-H, F-I, G-J, H-J, I-J. Implement functionalities primarily using Breadth-First Search (BFS) with required assumptions to answer the following queries: •□□□□□□ Given a source_device and a target_device, find the minimum number of hops (direct connections) required to establish a path between them. If no path exists, indicate so. •□□□□□□ Given a starting_device and a max_hops limit, determine all devices that can be reached from the starting_device within max_hops. This could represent the reach of a network broadcast or a virus. •□□□□□□ Given a specific pair of source_device and target_device, identify all "bridge devices" along every shortest path between them. A device is a "bridge device" in this context if its removal would increase the shortest path distance between the source_device and target_device or disconnect them entirely. This requires careful application or multiple runs of BFS. •□□□□□□ Given a starting_device, find the diameter of the connected component to which it belongs. The diameter is the longest shortest path between any two nodes in that component.	4 hours
4. Trees: AVL, Red-Black tree You are tasked with designing the core data structure for a large e-commerce platform's product catalog. Products are organized in a strict multi-level hierarchy (e.g., Electronics -> Laptops -> Gaming Laptops -> BrandX Gaming Laptop Model Y). Each product has a unique product_ID, a price, and a	4 hours

quantity_in_stock. The catalog is highly dynamic, with frequent additions, removals, price changes, and stock updates. Implement a robust and efficient tree-based data structure (e.g., a balanced Binary Search Tree like an AVL Tree or Red-Black Tree, where nodes can represent categories or individual products) to manage this hierarchical product catalog. Your implementation must support the CRUD operations.	
5. Divide & Conquer: Merge sort, Quick sort, Karatsuba's fast multiplication method You have n coins, all of which are gold except one coin which appears to be a gold coin, but it is fake. All gold coins are of the same weight, the fake coin weighs less than the others. You have a balance scale, you can put any number of coins on each side of the scale at one time and it will tell you if the two sides weight the same, or which side is lighter if they don't weight the same. The problem is to identify the fake coin. Design an algorithm to find the fake coin in the given n coins.	4 hours
6. Dynamic programming: MCM, LCS A chain of matrices A is called as Add-multiply-chain(AM-chain) if all the matrices of the chain are sqaure matrices of the same size and both addition & multiplication operations are involved in the chain. For example, $A = A_1 * A_2 * A_3 + A_4 * A_5 * A_6 * A_7 + A_8 * A_9 * A_{10}$ is an AM-chain. The above AM-chain A is written as (A, 10, 3, 7), which conveys that the chain A has 10 matrices, with two '+'operations, one '+'operation after A3, another '+' operation after A7, and multiplication is the operation in all the required places in the chain. Multiplication (or addition) between any two numbers is called a scalar operation. The number of scalar operations involved in the chain $A_1A_2 + A_3$ is 12, where all three matrices are square matrices of size 2. (A, n, i, j) is a chain A with n matrices with a '+' operation after A_i, another '+' operation after A_j, and all the other operations are multiplication. Given an AM-chain (A, n, i, j), design a pseudocode and implement it to parenthesize the chain A such that the number of scalar operations to evaluate the chain A is minimum.	2 hours
7. Backtracking: N-Queens problem, Subset Sum, Graph Coloring You have a standard 8x8 chessboard (if your are not familiar with game of chess, please get to know), empty but for a single knight on some square. Your task is to generate a series of legal knight moves that result in the knight visiting every square on the chessboard exactly once. In addition, the knight	2 hours

must end on a square that is one knight's move from the beginning square. The output of your program should indicate the order in which the knight visits the squares, starting with the initial position. Generalize your program for an n x n board where n>8.	
8. Branch & Bound: A-Star, Subset Sum You are developing the pathfinding core for a logistics company using autonomous delivery vehicles in a dense urban environment. The city map is represented as a grid, where each cell has an associated "traffic cost" (representing time delay due to congestion, road conditions, etc.). Some cells may be temporarily designated as "restricted zones" or "accident sites" with extremely high or infinite costs. Vehicles can move horizontally, vertically, and diagonally. You need to find the most cost-effective path from a starting depot to a delivery destination. Implement the A* search algorithm to find the minimum-cost path from a start_cell (start_row, start_col) to a target_cell (target_row, target_col) within a given M x N integer grid.	2 hours
9. Approximation Algorithm: Vertex Cover, Set Cover You are a network security engineer responsible for monitoring a critical communication network. The network consists of various communication nodes (e.g., servers, routers, switches) and the direct communication links between them. To detect potential intrusions or failures, you need to install monitoring agents on a subset of these nodes. An agent on a node can monitor all direct communication links connected to that node. Your goal is to select the minimum number of nodes to place agents on, such that every communication link in the network is monitored. However, finding the absolute minimum is a known NP-hard problem for large networks, so an efficient approximation is required. Implement a 2-approximation algorithm for the Minimum Vertex Cover problem.	4 hours
Total Laboratory Hours:	**30 hours**

Mode of Evaluation : Continuous Assessment Test, Final Assessment Test	
Recommended by Board of Studies :	23-05-2025
Approved by Academic Council : No. 78	12-06-2025